

APIs Used: [OpenWeatherMap API](#) (Free Tier)

1. The Data Sample (Save as **orders.json**)

"Please use this JSON as your local database. Your script must process these specific cities."

JSON

None

```
[
  {
    "order_id": "1001",
    "customer": "Alice Smith",
    "city": "New York",
    "status": "Pending"
  },
  {
    "order_id": "1002",
    "customer": "Bob Jones",
    "city": "Mumbai",
    "status": "Pending"
  },
  {
    "order_id": "1003",
    "customer": "Charlie Green",
    "city": "London",
    "status": "Pending"
  },
  {
    "order_id": "1004",
    "customer": "InvalidCity123",
    "city": "InvalidCity123",
    "status": "Pending"
  }
]
```

2. The Mission

Build a script that checks the weather for each order's city and flags potential delivery delays.

Requirements:

1. **Parallel Fetching:** Loop through the `orders.json` and fetch the current weather for each city using the OpenWeatherMap API.
 - **Constraint:** You must trigger these API calls **concurrently** (e.g., `Promise.all` in JS or `asyncio.gather` in Python), not one by one.
2. **The "Golden Flow" Logic:**
 - If the weather "main" status is **Rain**, **Snow**, or **Extreme**, update the order status to `Delayed` in the JSON.
 - **AI Challenge:** Use an AI tool to write a "Weather-Aware Apology" function. The script should generate a personalized message like: *"Hi Alice, your order to New York is delayed due to heavy rain. We appreciate your patience!"*
3. **Resilience:**
 - **Error Handling:** Your script must handle the `InvalidCity123` case. The error for that city should be logged, but the script **must not crash**—it should finish processing the other valid cities.
 - **Security:** Do not hardcode your API Key. Use a `.env` file.

3. Submission Requirements

- The source code (GitHub or ZIP).
- The updated `orders.json` (showing which orders were marked as `Delayed`).
- A short "AI Log" listing the prompts you used to build the **Parallel Fetching** and **Error Handling** logic.
- Demo recording